circuit

March 13, 2026

[95]:
```
from qiskit import *
from qiskit import QuantumCircuit
from qiskit.circuit import ParameterVector
from qiskit_aer import AerSimulator, StatevectorSimulator
from qiskit_aer.noise import NoiseModel, depolarizing_error
from scipy.optimize import minimize
import numpy as np
import matplotlib.pyplot as plt
```

[96]:
```
#                    Configuration
%matplotlib widget
# Dataset
N          = 1000    # Number of generated points
TEST_SIZE  = 0.2     # Fraction used for test set
SEED       = 1     # Random seed (data split & optimizer init)

# Model
RC         = 4       # Number of circuit layers

# Optimization
MAXITER    = 300     # COBYLA max iterations

# Simulation
USE_NOISE  = True   # True  → noisy AerSimulator  |  False Statevector
NOISE_RATE = 0.005   # Depolarizing error rate per gate
SHOTS      = 1024     # Number of shots
```

1 Data generation

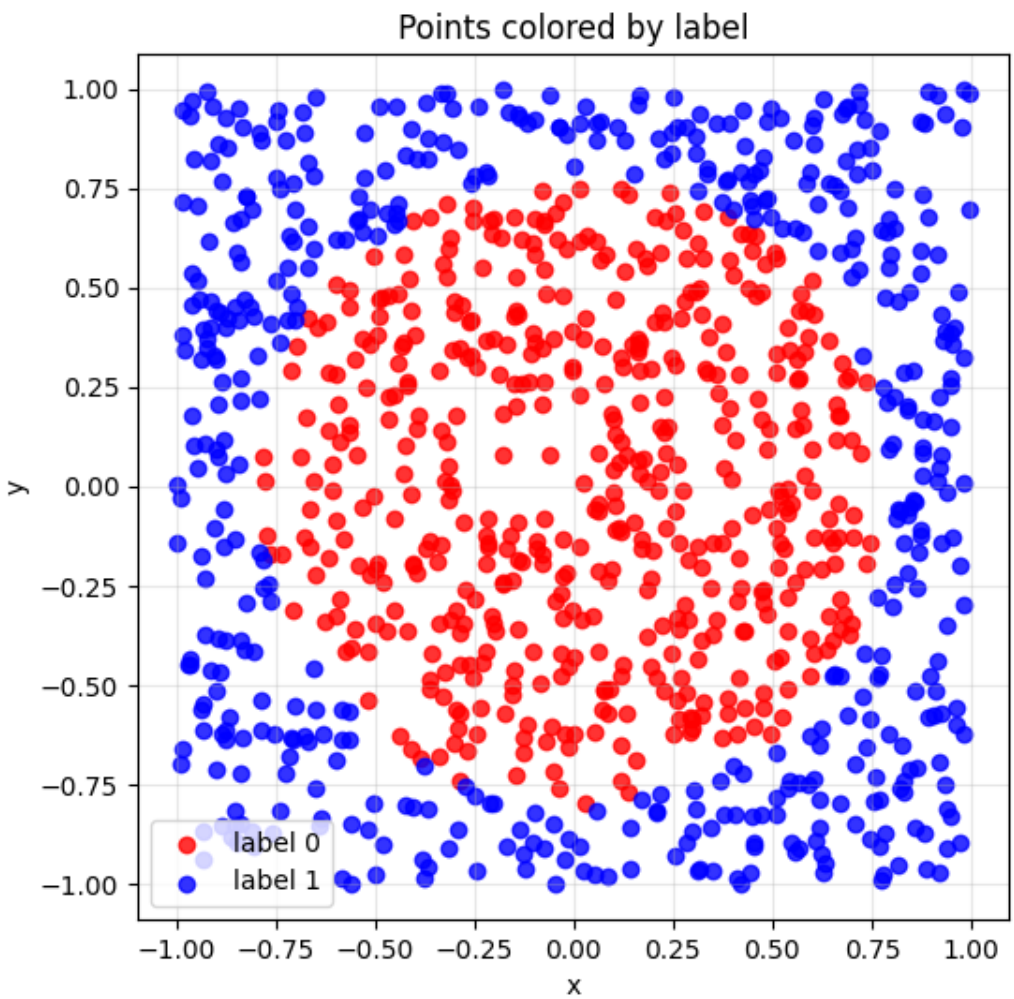
[97]:
```
points = np.random.rand(N, 2) * 2 - 1
labels = [0 if x**2 + y**2 < 2/np.pi else 1 for x, y in points]

arr = np.array(points)
lbl = np.array(labels)

plt.figure(figsize=(6, 6))
```

```
plt.scatter(arr[lbl == 0, 0], arr[lbl == 0, 1], c="red", label="label 0",␣
↪alpha=0.8)
plt.scatter(arr[lbl == 1, 0], arr[lbl == 1, 1], c="blue", label="label 1",␣
↪alpha=0.8)
plt.xlabel("x")
plt.ylabel("y")
plt.title("Points colored by label")
plt.axis("equal")
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

print("points in label 0:", np.sum(lbl == 0))
print("points in label 1:", np.sum(lbl == 1))
```



```
points in label 0: 506
points in label 1: 494
```

[98]:
```
rng = np.random.default_rng(SEED)

indices   = rng.permutation(len(arr))
split_idx = int(len(arr) * (1 - TEST_SIZE))

train_idx, test_idx = indices[:split_idx], indices[split_idx:]

X_train, X_test = arr[train_idx], arr[test_idx]
y_train, y_test = lbl[train_idx], lbl[test_idx]

print("X_train:", X_train.shape, "y_train:", y_train.shape)
print("X_test :", X_test.shape,  "y_test :", y_test.shape)
```

```
X_train: (800, 2) y_train: (800,)
X_test : (200, 2) y_test : (200,)
```

[99]:
```
def U_su2(q, theta, omega, x, qubit):
    x3d = [x[0], x[1], 0] # Add a third dimension with value 0
    # phi_i = theta_i + w_i * x_i
    val_0 = theta[0] + omega[0] * x3d[0]
    val_1 = theta[1] + omega[1] * x3d[1]
    val_2 = theta[2] + omega[2] * x3d[2]

    q.u(val_0, val_1, val_2, qubit)
```

[100]:
```
def create_circuit(x, theta, omega, measure=True):
    qc = QuantumCircuit(1)
    for i in range(RC):
        qc.h(0)
        U_su2(qc, theta[i], omega[i], x, 0)
    if measure:
        qc.measure_all()
    return qc
```

[101]:
```
theta = [[1, 1, 1] for _ in range(RC)]
omega = [[1, 1, 1] for _ in range(RC)]
qc = create_circuit(X_train[0], theta, omega)
qc.draw()
```

[101]:
```
                                      »
     q:   H   U(1.3719,1.409,1)   H   U(1.3719,1.409,1)   H »
                                      »
meas: 1/                              »
                                                            »
```

```
«
«     q:   U(1.3719,1.409,1)   H   U(1.3719,1.409,1)    M
«
«meas: 1/
«                                                        0
```

[102]:
```python
def build_simulator():
    if USE_NOISE:
        nm = NoiseModel()
        nm.add_all_qubit_quantum_error(depolarizing_error(NOISE_RATE, 1), ['u'])
        return AerSimulator(noise_model=nm)
    return StatevectorSimulator()

sv_sim = build_simulator()

def get_probs_batch(circuits):
    """
    Returns a list of (p0, p1) for each circuit.
    Noiseless: exact statevector amplitudes.
    Noisy:     shot-based counts normalised by SHOTS.
    """
    if USE_NOISE:
        meas = [c.copy() for c in circuits]
        for c in meas:
            c.measure_all()
        compiled = transpile(meas, sv_sim)
        results  = sv_sim.run(compiled, shots=SHOTS).result()
        return [
            (results.get_counts(i).get('0', 0) / SHOTS,
             results.get_counts(i).get('1', 0) / SHOTS)
            for i in range(len(meas))
        ]
    else:
        compiled = transpile(circuits, sv_sim)
        results  = sv_sim.run(compiled).result()
        return [
            (float(np.abs(results.get_statevector(i).data[0])**2),
             float(np.abs(results.get_statevector(i).data[1])**2))
            for i in range(len(circuits))
        ]
```

[103]:
```python
def cost_function_weighted(params, X_train, y_train):
    theta  = params[:3 * RC].reshape(RC, 3)
    omega  = params[3 * RC:6 * RC].reshape(RC, 3)
    alphas = params[6 * RC:]   # alpha_0, alpha_1

    circuits = [create_circuit(x, theta, omega, measure=False) for x in X_train]
```

```
    probs    = get_probs_batch(circuits)

    total_cost = 0.0
    for i, y_target in enumerate(y_train):
        f0, f1     = probs[i]
        y_expected = [1.0, 0.0] if y_target == 0 else [0.0, 1.0]
        total_cost += (alphas[0] * f0 - y_expected[0])**2
        total_cost += (alphas[1] * f1 - y_expected[1])**2

    return 0.5 * (total_cost / len(y_train))
```

[104]:
```
def unpack_params(params):
    theta  = params[:3 * RC].reshape(RC, 3)
    omega  = params[3 * RC:6 * RC].reshape(RC, 3)
    alphas = params[6 * RC:]
    return theta, omega, alphas

def optimize_parameters(X_train, y_train):
    rng  = np.random.default_rng(SEED)
    init = rng.uniform(-np.pi, np.pi, size=6 * RC + 2)

    cache = {"cost": None}
    cost_history = []
    step = {"k": 0}

    def objective(params):
        cost = cost_function_weighted(params, X_train, y_train)
        cache["cost"] = cost
        return cost

    def cb(xk):
        step["k"] += 1
        cost_history.append(cache["cost"])
        print(f"Step {step['k']:03d} | cost = {cache['cost']:.6f}")

    res = minimize(objective, init, method="COBYLA",
                   callback=cb, options={"maxiter": MAXITER})

    theta_opt, omega_opt, _ = unpack_params(res.x)
    print(f"Final cost = {res.fun:.6f}")
    return theta_opt, omega_opt, res, cost_history
```

[105]:
```
def predict_batch(X, theta, omega):
    circuits = [create_circuit(x, theta, omega, measure=False) for x in X]
    probs    = get_probs_batch(circuits)
    return np.array([0 if p0 >= 0.5 else 1 for p0, _ in probs])
```

```python
def evaluate_metrics(X, y, theta, omega, positive_label=1):
    y_pred    = predict_batch(X, theta, omega)
    tp        = np.sum((y_pred == positive_label) & (y == positive_label))
    fp        = np.sum((y_pred == positive_label) & (y != positive_label))
    precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
    accuracy  = np.mean(y_pred == y)
    return precision, accuracy, y_pred

# --- train + test (noisy run) ---
original_use_noise = USE_NOISE

USE_NOISE = True
sv_sim = build_simulator()
theta_opt_noise, omega_opt_noise, res_noise, cost_history_noise =␣
 ↪optimize_parameters(X_train, y_train)
precision_test_noise, accuracy_test_noise, y_pred_noise = evaluate_metrics(
    X_test, y_test, theta_opt_noise, omega_opt_noise
)

print("[Noisy] Final train cost:", res_noise.fun)
print("[Noisy] Test precision:",   precision_test_noise)
print("[Noisy] Test accuracy: ",   accuracy_test_noise)

# --- train + test (noiseless run) ---
USE_NOISE = False
sv_sim = build_simulator()
theta_opt_noiseless, omega_opt_noiseless, res_noiseless, cost_history_noiseless␣
 ↪= optimize_parameters(X_train, y_train)
precision_test_noiseless, accuracy_test_noiseless, y_pred_noiseless =␣
 ↪evaluate_metrics(
    X_test, y_test, theta_opt_noiseless, omega_opt_noiseless
)

print("[Noiseless] Final train cost:", res_noiseless.fun)
print("[Noiseless] Test precision:",   precision_test_noiseless)
print("[Noiseless] Test accuracy: ",   accuracy_test_noiseless)

# Keep default behavior for following cells
USE_NOISE = original_use_noise
sv_sim = build_simulator()

# Reuse previous variable names for compatibility with downstream cells
theta_opt = theta_opt_noise
omega_opt = omega_opt_noise
res = res_noise
cost_history = cost_history_noise
precision_test = precision_test_noise
```

```
accuracy_test = accuracy_test_noise
y_pred = y_pred_noise
```

```
Step 001 | cost = 1.260894
Step 002 | cost = 1.461371
Step 003 | cost = 0.883242
Step 004 | cost = 0.694137
Step 005 | cost = 0.607902
Step 006 | cost = 0.709612
Step 007 | cost = 1.108824
Step 008 | cost = 0.553293
Step 009 | cost = 0.955046
Step 010 | cost = 0.698523
Step 011 | cost = 0.800680
Step 012 | cost = 0.553073
Step 013 | cost = 0.593596
Step 014 | cost = 0.556296
Step 015 | cost = 0.577299
Step 016 | cost = 0.553423
Step 017 | cost = 0.663948
Step 018 | cost = 0.648242
Step 019 | cost = 0.554868
Step 020 | cost = 0.553596
Step 021 | cost = 0.606539
Step 022 | cost = 0.555414
Step 023 | cost = 0.491648
Step 024 | cost = 0.488549
Step 025 | cost = 0.492179
Step 026 | cost = 0.682583
Step 027 | cost = 1.119979
Step 028 | cost = 0.494337
Step 029 | cost = 0.578263
Step 030 | cost = 0.634010
Step 031 | cost = 0.571803
Step 032 | cost = 0.665566
Step 033 | cost = 0.573101
Step 034 | cost = 0.648156
Step 035 | cost = 0.487662
Step 036 | cost = 0.461084
Step 037 | cost = 0.405935
Step 038 | cost = 0.992881
Step 039 | cost = 0.442295
Step 040 | cost = 0.451660
Step 041 | cost = 0.510624
Step 042 | cost = 0.438501
Step 043 | cost = 0.401927
Step 044 | cost = 0.406128
Step 045 | cost = 0.426211
```

```
Step 046 | cost = 0.333524
Step 047 | cost = 0.334574
Step 048 | cost = 0.307827
Step 049 | cost = 0.297569
Step 050 | cost = 0.466305
Step 051 | cost = 0.276237
Step 052 | cost = 0.759292
Step 053 | cost = 0.429127
Step 054 | cost = 0.339680
Step 055 | cost = 0.395885
Step 056 | cost = 0.336283
Step 057 | cost = 0.275774
Step 058 | cost = 0.347333
Step 059 | cost = 0.276632
Step 060 | cost = 0.477471
Step 061 | cost = 0.308493
Step 062 | cost = 0.338914
Step 063 | cost = 0.240553
Step 064 | cost = 0.573200
Step 065 | cost = 0.251772
Step 066 | cost = 0.403818
Step 067 | cost = 0.284673
Step 068 | cost = 0.334821
Step 069 | cost = 0.266422
Step 070 | cost = 0.246765
Step 071 | cost = 0.248946
Step 072 | cost = 0.239986
Step 073 | cost = 0.264574
Step 074 | cost = 0.238644
Step 075 | cost = 0.246348
Step 076 | cost = 0.238450
Step 077 | cost = 0.258133
Step 078 | cost = 0.240083
Step 079 | cost = 0.339383
Step 080 | cost = 0.219453
Step 081 | cost = 0.281908
Step 082 | cost = 0.220784
Step 083 | cost = 0.234050
Step 084 | cost = 0.218588
Step 085 | cost = 0.272333
Step 086 | cost = 0.231641
Step 087 | cost = 0.319664
Step 088 | cost = 0.223764
Step 089 | cost = 0.266873
Step 090 | cost = 0.229884
Step 091 | cost = 0.223339
Step 092 | cost = 0.222117
Step 093 | cost = 0.226902
```

```
Step 094 | cost = 0.213689
Step 095 | cost = 0.215057
Step 096 | cost = 0.213054
Step 097 | cost = 0.219037
Step 098 | cost = 0.210249
Step 099 | cost = 0.205177
Step 100 | cost = 0.213014
Step 101 | cost = 0.224621
Step 102 | cost = 0.204526
Step 103 | cost = 0.220019
Step 104 | cost = 0.205352
Step 105 | cost = 0.211415
Step 106 | cost = 0.197261
Step 107 | cost = 0.198669
Step 108 | cost = 0.197137
Step 109 | cost = 0.208776
Step 110 | cost = 0.194851
Step 111 | cost = 0.187579
Step 112 | cost = 0.187319
Step 113 | cost = 0.194562
Step 114 | cost = 0.197279
Step 115 | cost = 0.184814
Step 116 | cost = 0.186378
Step 117 | cost = 0.198136
Step 118 | cost = 0.186130
Step 119 | cost = 0.194189
Step 120 | cost = 0.185256
Step 121 | cost = 0.189599
Step 122 | cost = 0.184862
Step 123 | cost = 0.183290
Step 124 | cost = 0.184893
Step 125 | cost = 0.188749
Step 126 | cost = 0.184951
Step 127 | cost = 0.188974
Step 128 | cost = 0.190108
Step 129 | cost = 0.184655
Step 130 | cost = 0.185382
Step 131 | cost = 0.178024
Step 132 | cost = 0.187525
Step 133 | cost = 0.175484
Step 134 | cost = 0.175729
Step 135 | cost = 0.184765
Step 136 | cost = 0.179469
Step 137 | cost = 0.194913
Step 138 | cost = 0.172766
Step 139 | cost = 0.191199
Step 140 | cost = 0.169577
Step 141 | cost = 0.168443
```

```
Step 142 | cost = 0.168603
Step 143 | cost = 0.172859
Step 144 | cost = 0.170588
Step 145 | cost = 0.173910
Step 146 | cost = 0.168655
Step 147 | cost = 0.178251
Step 148 | cost = 0.172736
Step 149 | cost = 0.165959
Step 150 | cost = 0.162569
Step 151 | cost = 0.166074
Step 152 | cost = 0.170051
Step 153 | cost = 0.169416
Step 154 | cost = 0.167849
Step 155 | cost = 0.166620
Step 156 | cost = 0.160666
Step 157 | cost = 0.161177
Step 158 | cost = 0.169765
Step 159 | cost = 0.176439
Step 160 | cost = 0.159124
Step 161 | cost = 0.157540
Step 162 | cost = 0.157768
Step 163 | cost = 0.151035
Step 164 | cost = 0.156338
Step 165 | cost = 0.155397
Step 166 | cost = 0.152951
Step 167 | cost = 0.165703
Step 168 | cost = 0.150752
Step 169 | cost = 0.154867
Step 170 | cost = 0.153125
Step 171 | cost = 0.146379
Step 172 | cost = 0.143960
Step 173 | cost = 0.150797
Step 174 | cost = 0.144120
Step 175 | cost = 0.146568
Step 176 | cost = 0.143733
Step 177 | cost = 0.139764
Step 178 | cost = 0.149304
Step 179 | cost = 0.145911
Step 180 | cost = 0.141486
Step 181 | cost = 0.129568
Step 182 | cost = 0.129682
Step 183 | cost = 0.130368
Step 184 | cost = 0.131784
Step 185 | cost = 0.130875
Step 186 | cost = 0.136121
Step 187 | cost = 0.129695
Step 188 | cost = 0.143137
Step 189 | cost = 0.133924
```

```
Step 190 | cost = 0.130511
Step 191 | cost = 0.131469
Step 192 | cost = 0.131509
Step 193 | cost = 0.130184
Step 194 | cost = 0.128448
Step 195 | cost = 0.125778
Step 196 | cost = 0.125129
Step 197 | cost = 0.123952
Step 198 | cost = 0.129492
Step 199 | cost = 0.128250
Step 200 | cost = 0.120959
Step 201 | cost = 0.122536
Step 202 | cost = 0.123334
Step 203 | cost = 0.121555
Step 204 | cost = 0.122181
Step 205 | cost = 0.124860
Step 206 | cost = 0.117015
Step 207 | cost = 0.121086
Step 208 | cost = 0.117298
Step 209 | cost = 0.125509
Step 210 | cost = 0.119378
Step 211 | cost = 0.115718
Step 212 | cost = 0.116586
Step 213 | cost = 0.115843
Step 214 | cost = 0.117666
Step 215 | cost = 0.124695
Step 216 | cost = 0.115671
Step 217 | cost = 0.118460
Step 218 | cost = 0.116948
Step 219 | cost = 0.119643
Step 220 | cost = 0.123804
Step 221 | cost = 0.112538
Step 222 | cost = 0.115738
Step 223 | cost = 0.114467
Step 224 | cost = 0.115278
Step 225 | cost = 0.113852
Step 226 | cost = 0.114813
Step 227 | cost = 0.111775
Step 228 | cost = 0.113195
Step 229 | cost = 0.117080
Step 230 | cost = 0.113477
Step 231 | cost = 0.110406
Step 232 | cost = 0.110566
Step 233 | cost = 0.112014
Step 234 | cost = 0.110102
Step 235 | cost = 0.117003
Step 236 | cost = 0.110314
Step 237 | cost = 0.114024
```

```
Step 238 | cost = 0.111626
Step 239 | cost = 0.109217
Step 240 | cost = 0.109908
Step 241 | cost = 0.111460
Step 242 | cost = 0.108482
Step 243 | cost = 0.119095
Step 244 | cost = 0.110074
Step 245 | cost = 0.109157
Step 246 | cost = 0.107565
Step 247 | cost = 0.108412
Step 248 | cost = 0.109390
Step 249 | cost = 0.115934
Step 250 | cost = 0.106971
Step 251 | cost = 0.106458
Step 252 | cost = 0.105179
Step 253 | cost = 0.112262
Step 254 | cost = 0.105863
Step 255 | cost = 0.105912
Step 256 | cost = 0.105797
Step 257 | cost = 0.110079
Step 258 | cost = 0.104783
Step 259 | cost = 0.108556
Step 260 | cost = 0.104172
Step 261 | cost = 0.106683
Step 262 | cost = 0.104588
Step 263 | cost = 0.109680
Step 264 | cost = 0.106236
Step 265 | cost = 0.110175
Step 266 | cost = 0.107171
Step 267 | cost = 0.105012
Step 268 | cost = 0.107756
Step 269 | cost = 0.104829
Step 270 | cost = 0.105471
Step 271 | cost = 0.105263
Step 272 | cost = 0.105149
Step 273 | cost = 0.105281
Step 274 | cost = 0.106537
Step 275 | cost = 0.104298
Step 276 | cost = 0.106071
Step 277 | cost = 0.106578
Step 278 | cost = 0.107106
Step 279 | cost = 0.104714
Step 280 | cost = 0.106179
Step 281 | cost = 0.105049
Step 282 | cost = 0.105360
Step 283 | cost = 0.105159
Step 284 | cost = 0.104664
Step 285 | cost = 0.105042
```

```
Step 286 | cost = 0.104140
Step 287 | cost = 0.103810
Step 288 | cost = 0.104145
Step 289 | cost = 0.104278
Step 290 | cost = 0.104312
Step 291 | cost = 0.104246
Step 292 | cost = 0.104019
Step 293 | cost = 0.104434
Step 294 | cost = 0.104395
Step 295 | cost = 0.104942
Step 296 | cost = 0.104923
Step 297 | cost = 0.104229
Step 298 | cost = 0.103310
Step 299 | cost = 0.103772
Step 300 | cost = 0.103823
Step 301 | cost = 0.103823
Final cost = 0.103310
[Noisy] Final train cost: 0.10331019220399444
[Noisy] Test precision: 0.948051948051948
[Noisy] Test accuracy:  0.845
Step 001 | cost = 1.266979
Step 002 | cost = 1.469923
Step 003 | cost = 0.884492
Step 004 | cost = 0.693552
Step 005 | cost = 0.605931
Step 006 | cost = 0.710667
Step 007 | cost = 1.113026
Step 008 | cost = 0.553564
Step 009 | cost = 0.955359
Step 010 | cost = 0.703163
Step 011 | cost = 0.801123
Step 012 | cost = 0.553564
Step 013 | cost = 0.594711
Step 014 | cost = 0.554555
Step 015 | cost = 0.577121
Step 016 | cost = 0.553564
Step 017 | cost = 0.665460
Step 018 | cost = 0.652299
Step 019 | cost = 0.553564
Step 020 | cost = 0.551994
Step 021 | cost = 0.643924
Step 022 | cost = 0.551994
Step 023 | cost = 0.435901
Step 024 | cost = 0.435901
Step 025 | cost = 0.435901
Step 026 | cost = 0.582367
Step 027 | cost = 1.103292
Step 028 | cost = 0.339179
```

```
Step 029 | cost = 0.358328
Step 030 | cost = 0.663152
Step 031 | cost = 0.495004
Step 032 | cost = 0.399755
Step 033 | cost = 0.281371
Step 034 | cost = 0.648035
Step 035 | cost = 0.437052
Step 036 | cost = 0.343483
Step 037 | cost = 0.339884
Step 038 | cost = 0.394056
Step 039 | cost = 0.332591
Step 040 | cost = 0.385820
Step 041 | cost = 0.289673
Step 042 | cost = 0.303884
Step 043 | cost = 0.266595
Step 044 | cost = 0.290570
Step 045 | cost = 0.266595
Step 046 | cost = 0.619395
Step 047 | cost = 0.266595
Step 048 | cost = 0.284962
Step 049 | cost = 0.266595
Step 050 | cost = 0.322372
Step 051 | cost = 0.320241
Step 052 | cost = 0.308972
Step 053 | cost = 0.297119
Step 054 | cost = 0.417398
Step 055 | cost = 0.309272
Step 056 | cost = 0.266595
Step 057 | cost = 0.250046
Step 058 | cost = 0.250046
Step 059 | cost = 0.289749
Step 060 | cost = 0.299388
Step 061 | cost = 0.282312
Step 062 | cost = 0.276587
Step 063 | cost = 0.244653
Step 064 | cost = 0.272822
Step 065 | cost = 0.240955
Step 066 | cost = 0.272345
Step 067 | cost = 0.247618
Step 068 | cost = 0.236773
Step 069 | cost = 0.235817
Step 070 | cost = 0.240493
Step 071 | cost = 0.323482
Step 072 | cost = 0.237880
Step 073 | cost = 0.370458
Step 074 | cost = 0.233461
Step 075 | cost = 0.251374
Step 076 | cost = 0.223356
```

```
Step 077 | cost = 0.257307
Step 078 | cost = 0.215249
Step 079 | cost = 0.259186
Step 080 | cost = 0.215249
Step 081 | cost = 0.273136
Step 082 | cost = 0.215249
Step 083 | cost = 0.286090
Step 084 | cost = 0.215249
Step 085 | cost = 0.223532
Step 086 | cost = 0.215249
Step 087 | cost = 0.243422
Step 088 | cost = 0.215249
Step 089 | cost = 0.240399
Step 090 | cost = 0.225877
Step 091 | cost = 0.214070
Step 092 | cost = 0.223667
Step 093 | cost = 0.217049
Step 094 | cost = 0.219033
Step 095 | cost = 0.215793
Step 096 | cost = 0.226403
Step 097 | cost = 0.212447
Step 098 | cost = 0.236673
Step 099 | cost = 0.214005
Step 100 | cost = 0.229162
Step 101 | cost = 0.215148
Step 102 | cost = 0.226226
Step 103 | cost = 0.215781
Step 104 | cost = 0.223730
Step 105 | cost = 0.220440
Step 106 | cost = 0.207925
Step 107 | cost = 0.204321
Step 108 | cost = 0.224181
Step 109 | cost = 0.223039
Step 110 | cost = 0.196911
Step 111 | cost = 0.194513
Step 112 | cost = 0.198732
Step 113 | cost = 0.194333
Step 114 | cost = 0.199979
Step 115 | cost = 0.194088
Step 116 | cost = 0.180286
Step 117 | cost = 0.179725
Step 118 | cost = 0.178720
Step 119 | cost = 0.209475
Step 120 | cost = 0.177178
Step 121 | cost = 0.183372
Step 122 | cost = 0.173334
Step 123 | cost = 0.172002
Step 124 | cost = 0.174134
```

```
Step 125 | cost = 0.173283
Step 126 | cost = 0.171436
Step 127 | cost = 0.174738
Step 128 | cost = 0.178623
Step 129 | cost = 0.172053
Step 130 | cost = 0.172397
Step 131 | cost = 0.171741
Step 132 | cost = 0.174943
Step 133 | cost = 0.174900
Step 134 | cost = 0.175109
Step 135 | cost = 0.179750
Step 136 | cost = 0.178168
Step 137 | cost = 0.186549
Step 138 | cost = 0.170410
Step 139 | cost = 0.172806
Step 140 | cost = 0.167787
Step 141 | cost = 0.178049
Step 142 | cost = 0.167503
Step 143 | cost = 0.172704
Step 144 | cost = 0.172241
Step 145 | cost = 0.175003
Step 146 | cost = 0.166653
Step 147 | cost = 0.166844
Step 148 | cost = 0.167683
Step 149 | cost = 0.166587
Step 150 | cost = 0.168697
Step 151 | cost = 0.166627
Step 152 | cost = 0.166300
Step 153 | cost = 0.167669
Step 154 | cost = 0.166645
Step 155 | cost = 0.170420
Step 156 | cost = 0.163956
Step 157 | cost = 0.163281
Step 158 | cost = 0.169071
Step 159 | cost = 0.163339
Step 160 | cost = 0.161631
Step 161 | cost = 0.161255
Step 162 | cost = 0.161371
Step 163 | cost = 0.164009
Step 164 | cost = 0.161169
Step 165 | cost = 0.164093
Step 166 | cost = 0.157666
Step 167 | cost = 0.160898
Step 168 | cost = 0.156091
Step 169 | cost = 0.156127
Step 170 | cost = 0.155277
Step 171 | cost = 0.156933
Step 172 | cost = 0.155300
```

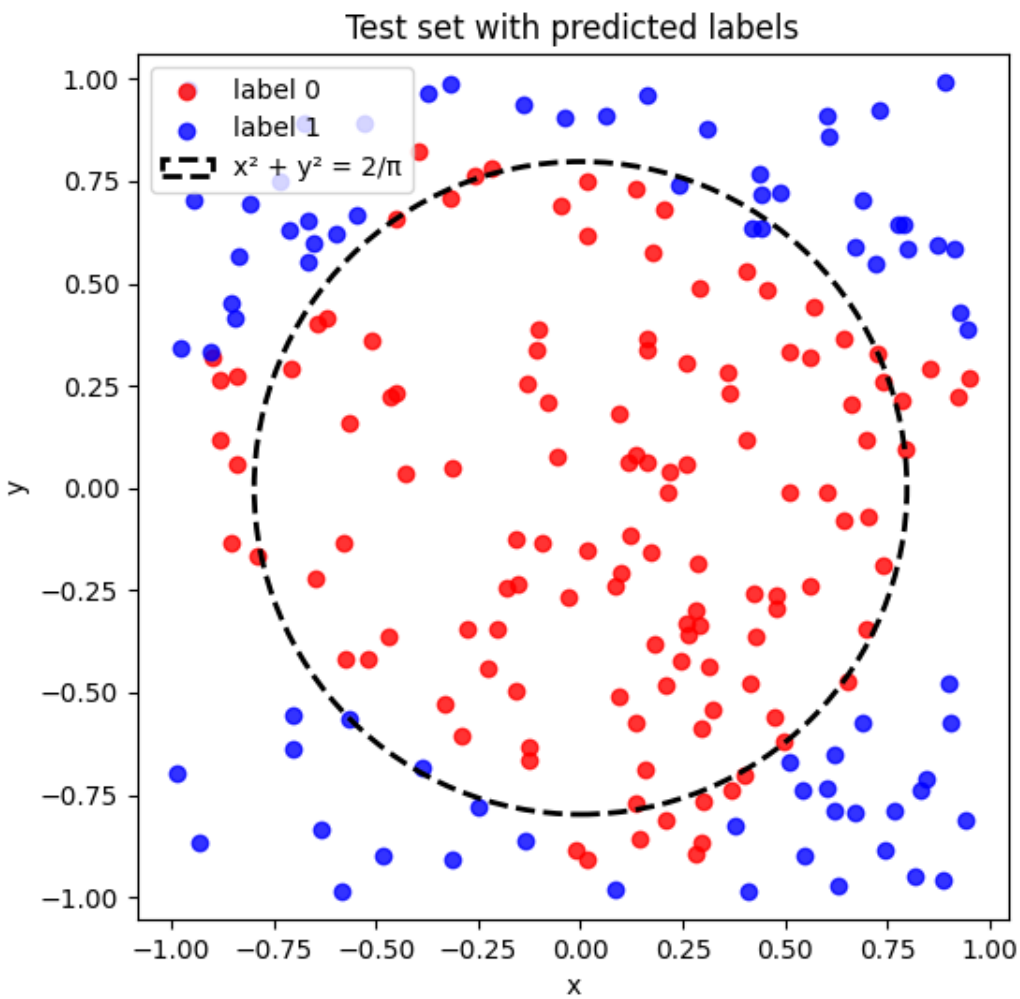
```
Step 173 | cost = 0.155092
Step 174 | cost = 0.154789
Step 175 | cost = 0.155711
Step 176 | cost = 0.154968
Step 177 | cost = 0.155588
Step 178 | cost = 0.154804
Step 179 | cost = 0.157686
Step 180 | cost = 0.155456
Step 181 | cost = 0.157172
Step 182 | cost = 0.156897
Step 183 | cost = 0.156672
Step 184 | cost = 0.154686
Step 185 | cost = 0.156388
Step 186 | cost = 0.155507
Step 187 | cost = 0.155831
Step 188 | cost = 0.155049
Step 189 | cost = 0.159095
Step 190 | cost = 0.155812
Step 191 | cost = 0.157509
Step 192 | cost = 0.153825
Step 193 | cost = 0.153806
Step 194 | cost = 0.155391
Step 195 | cost = 0.154176
Step 196 | cost = 0.152380
Step 197 | cost = 0.152046
Step 198 | cost = 0.151771
Step 199 | cost = 0.152230
Step 200 | cost = 0.151742
Step 201 | cost = 0.151551
Step 202 | cost = 0.152828
Step 203 | cost = 0.149058
Step 204 | cost = 0.148442
Step 205 | cost = 0.148434
Step 206 | cost = 0.146841
Step 207 | cost = 0.145698
Step 208 | cost = 0.145174
Step 209 | cost = 0.145886
Step 210 | cost = 0.145330
Step 211 | cost = 0.145248
Step 212 | cost = 0.144938
Step 213 | cost = 0.144893
Step 214 | cost = 0.144824
Step 215 | cost = 0.145332
Step 216 | cost = 0.143839
Step 217 | cost = 0.143967
Step 218 | cost = 0.142393
Step 219 | cost = 0.142217
Step 220 | cost = 0.144605
```

```
Step 221 | cost = 0.141776
Step 222 | cost = 0.140759
Step 223 | cost = 0.140530
Step 224 | cost = 0.140693
Step 225 | cost = 0.139709
Step 226 | cost = 0.138436
Step 227 | cost = 0.138164
Step 228 | cost = 0.138207
Step 229 | cost = 0.138401
Step 230 | cost = 0.138474
Step 231 | cost = 0.138521
Step 232 | cost = 0.138006
Step 233 | cost = 0.139977
Step 234 | cost = 0.137935
Step 235 | cost = 0.139571
Step 236 | cost = 0.139521
Step 237 | cost = 0.139095
Step 238 | cost = 0.138149
Step 239 | cost = 0.136054
Step 240 | cost = 0.135647
Step 241 | cost = 0.135412
Step 242 | cost = 0.135115
Step 243 | cost = 0.134548
Step 244 | cost = 0.135388
Step 245 | cost = 0.133923
Step 246 | cost = 0.133750
Step 247 | cost = 0.134692
Step 248 | cost = 0.133970
Step 249 | cost = 0.136053
Step 250 | cost = 0.133742
Step 251 | cost = 0.135085
Step 252 | cost = 0.133247
Step 253 | cost = 0.134809
Step 254 | cost = 0.133744
Step 255 | cost = 0.132211
Step 256 | cost = 0.131529
Step 257 | cost = 0.130947
Step 258 | cost = 0.131304
Step 259 | cost = 0.130250
Step 260 | cost = 0.130263
Step 261 | cost = 0.130533
Step 262 | cost = 0.130068
Step 263 | cost = 0.130615
Step 264 | cost = 0.130382
Step 265 | cost = 0.130235
Step 266 | cost = 0.130381
Step 267 | cost = 0.130766
Step 268 | cost = 0.129839
```

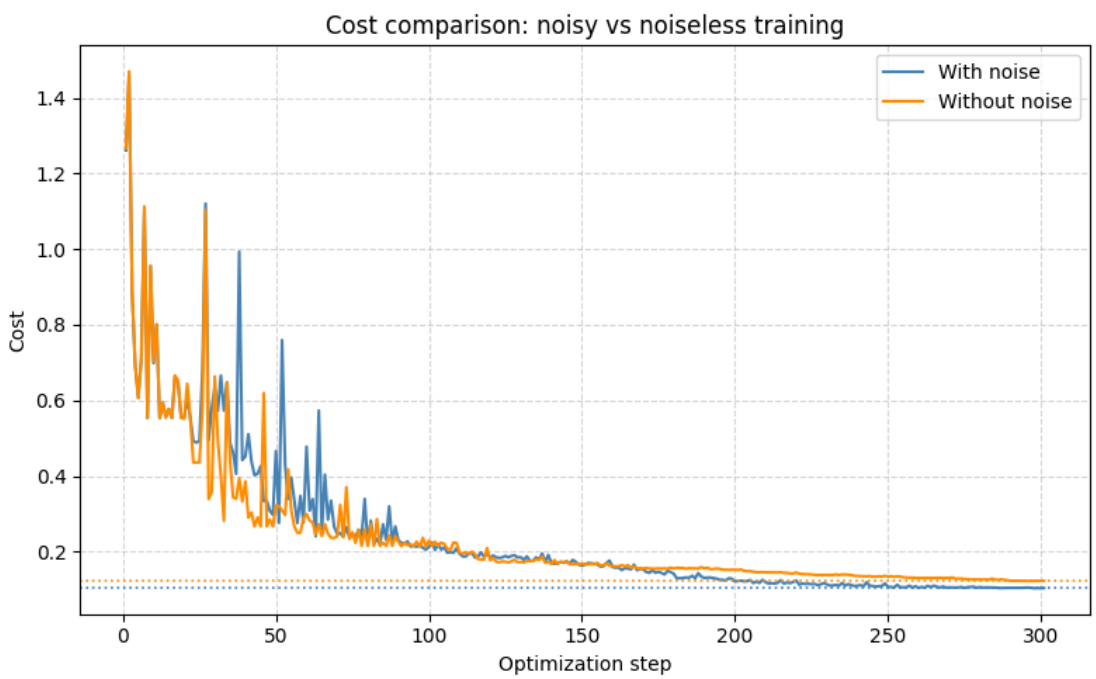
```
Step 269 | cost = 0.130718
Step 270 | cost = 0.130108
Step 271 | cost = 0.131768
Step 272 | cost = 0.128538
Step 273 | cost = 0.128536
Step 274 | cost = 0.128598
Step 275 | cost = 0.127442
Step 276 | cost = 0.127333
Step 277 | cost = 0.126035
Step 278 | cost = 0.126663
Step 279 | cost = 0.126035
Step 280 | cost = 0.126468
Step 281 | cost = 0.125994
Step 282 | cost = 0.126426
Step 283 | cost = 0.125861
Step 284 | cost = 0.128529
Step 285 | cost = 0.126422
Step 286 | cost = 0.125103
Step 287 | cost = 0.126594
Step 288 | cost = 0.125343
Step 289 | cost = 0.124611
Step 290 | cost = 0.123104
Step 291 | cost = 0.122972
Step 292 | cost = 0.122644
Step 293 | cost = 0.122698
Step 294 | cost = 0.122448
Step 295 | cost = 0.122296
Step 296 | cost = 0.122613
Step 297 | cost = 0.122219
Step 298 | cost = 0.122533
Step 299 | cost = 0.122398
Step 300 | cost = 0.122885
Step 301 | cost = 0.122885
Final cost = 0.122219
[Noiseless] Final train cost: 0.12221880263337143
[Noiseless] Test precision: 0.6950354609929078
[Noiseless] Test accuracy:  0.775
```

[106]:
```
plt.figure(figsize=(6, 6))
plt.scatter(X_test[y_pred == 0][:, 0], X_test[y_pred == 0][:, 1], c="red",␣
 ↪label="label 0", alpha=0.8)
plt.scatter(X_test[y_pred == 1][:, 0], X_test[y_pred == 1][:, 1], c="blue",␣
 ↪label="label 1", alpha=0.8)
r = np.sqrt(2 / np.pi)
circle = plt.Circle((0, 0), r, color="black", fill=False, linestyle="--",␣
 ↪linewidth=2, label="x² + y² = 2/ ")
plt.gca().add_patch(circle)
```

```
plt.axis("equal")
plt.xlabel("x")
plt.ylabel("y")
plt.title("Test set with predicted labels")
plt.legend()
plt.show()
```



[107]:
```
plt.figure(figsize=(8, 5))
ax1 = plt.subplot(1, 1, 1)

ax1.plot(
    range(1, len(cost_history_noise) + 1),
```

```
    cost_history_noise,
    color="steelblue",
    linewidth=1.5,
    label="With noise"
)
ax1.plot(
    range(1, len(cost_history_noiseless) + 1),
    cost_history_noiseless,
    color="darkorange",
    linewidth=1.5,
    label="Without noise"
)

ax1.set_xlabel("Optimization step")
ax1.set_ylabel("Cost")
ax1.set_title("Cost comparison: noisy vs noiseless training")
ax1.grid(True, linestyle="--", alpha=0.5)
ax1.axhline(res_noise.fun, color="steelblue", linestyle=":", linewidth=1.2)
ax1.axhline(res_noiseless.fun, color="darkorange", linestyle=":", linewidth=1.2)
ax1.legend()

plt.tight_layout()
plt.show()
```